Patrones GOF (Gang of Four):

¿Cómo se dividen los patrones GOF y qué significa cada categoría? Realizar una explicación de cada uno de ellos, con su respectivo diagrama de clases y un ejemplo, destacando cuándo es conveniente utilizarlos y en qué contexto.

Los 23 patrones GoF, publicados en el libro *Design Patterns* (1994), se dividen en tres categorías según el problema que resuelven: creacionales, estructurales y de comportamiento.

Categoría	Qué significa	Patrones
Creacionales	Abstraen el proceso de creación de objetos. Desacoplan al código que usa un objeto de la clase concreta y de cómo se construye.	Abstract Factory, Builder, Factory Method, Prototype, Singleton
Estructurales	Explican cómo combinar clases y objetos para formar estructuras mayores, manteniéndolas flexibles y eficientes.	Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
De comportamiento	Definen cómo se reparten las responsabilidades y cómo se comunican los objetos entre sí.	Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

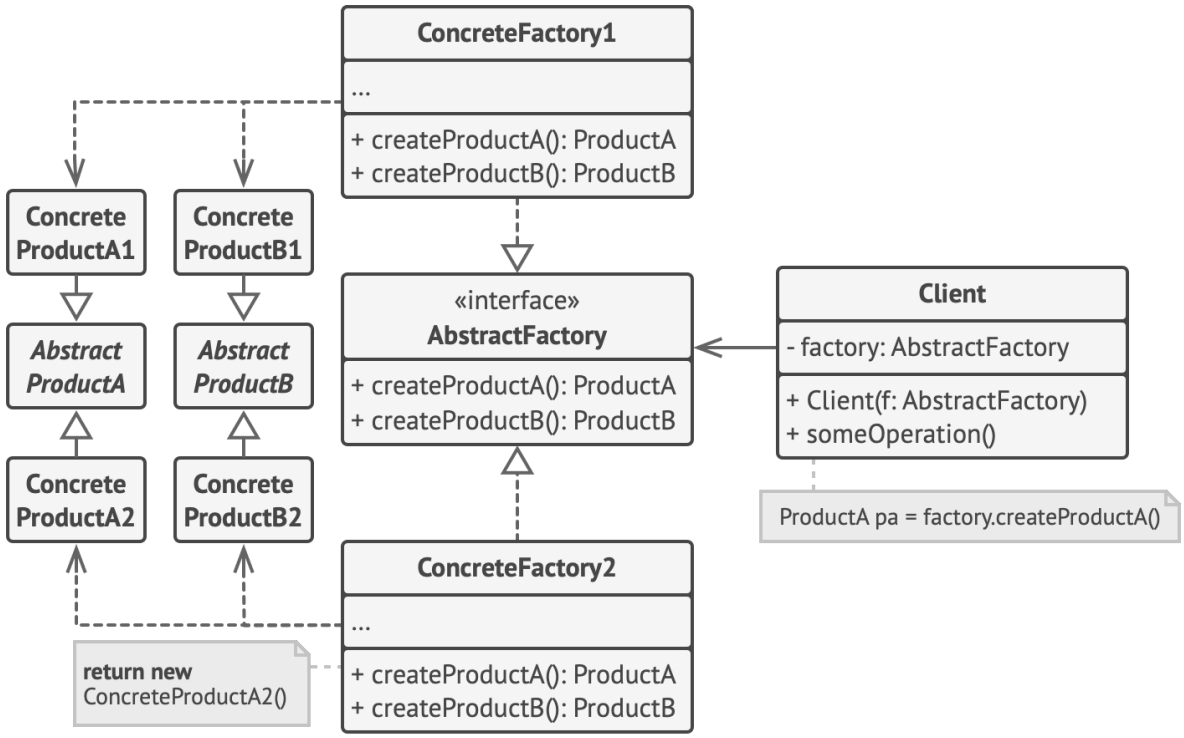
En cada patrón que sigue se indica qué problema resuelve, su diagrama de clases, un ejemplo en Java, cuándo conviene usarlo y en qué contexto aparece.

Patrones creacionales

Resuelven el problema de crear objetos sin acoplar el código a clases concretas.

Abstract Factory (Fábrica abstracta)

Proporciona una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas.



fuente: Refactoring Guru

Ejemplo en Java: acceso a base de datos según el motor.

```
interface BaseDatosFactory {
    Conexion crearConexion();
    Comando crearComando();
}
class MySQLFactory implements BaseDatosFactory {
    public Conexion crearConexion() { return new MySQLConexion(); }
    public Comando crearComando()   { return new MySQLComando(); }
}
class PostgresFactory implements BaseDatosFactory { /* análoga, con las clases de
Postgres */ }

BaseDatosFactory f = new PostgresFactory();   // se elige una vez, por configuración
Conexion c = f.crearConexion();
Comando q = f.crearComando();
```

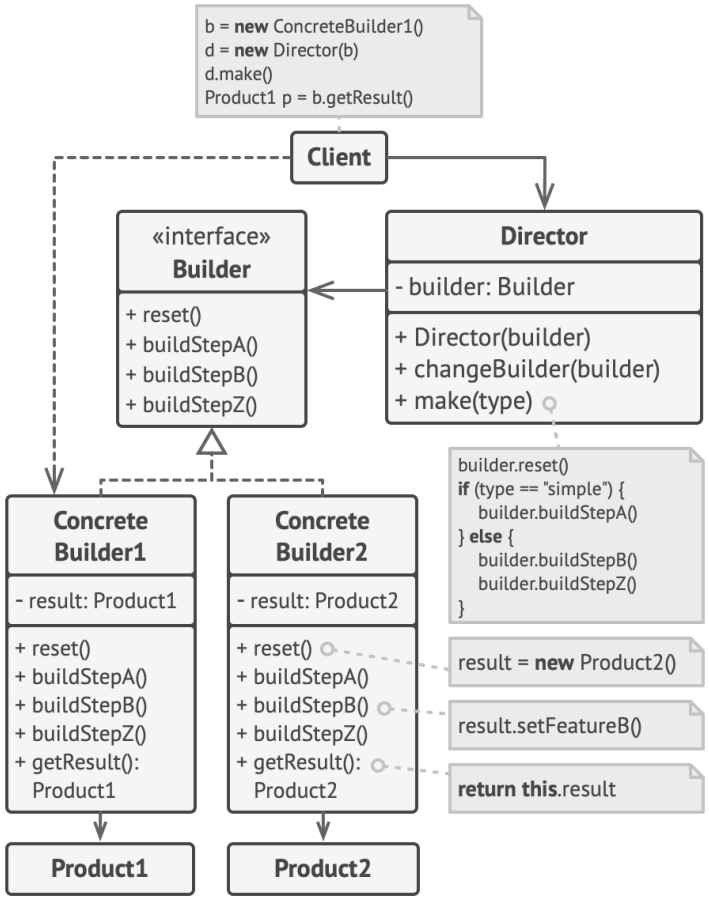
Roles en el ejemplo: BaseDatosFactory es la AbstractFactory; MySQLFactory y PostgresFactory son las ConcreteFactory; Conexion y Comando son los AbstractProduct; MySQLConexion, MySQLComando y sus equivalentes de Postgres son los ConcreteProduct.

- **Cuándo usarlo:** cuando hay varias familias de productos que deben usarse juntas y querés cambiar la familia completa tocando un solo punto.

- **Contexto:** acceso a bases de datos según el motor, interfaces gráficas multiplataforma, temas visuales.

Builder (Constructor)

Separa la construcción de un objeto complejo de su representación, para crearlo paso a paso.



fuente: Refactoring Guru

Ejemplo en Java: armado de una pizza con ingredientes opcionales.

```
class Pizza {
    private final String tamanio;
    private final List<String> ingredientes;
    private final boolean bordeRelleno;

    private Pizza(Builder b) {
        tamanio = b.tamanio; ingredientes = b.ingredientes; bordeRelleno =
b.bordeRelleno;
    }
```

```
    static class Builder {
        private String tamanio = "mediana";
        private List<String> ingredientes = new ArrayList<>();
        private boolean bordeRelleno;

        Builder tamanio(String t)        { tamanio = t; return this; }
        Builder ingrediente(String i)    { ingredientes.add(i); return this; }
        Builder bordeRelleno(boolean b)  { bordeRelleno = b; return this; }
        Pizza build()                    { return new Pizza(this); }
    }
}

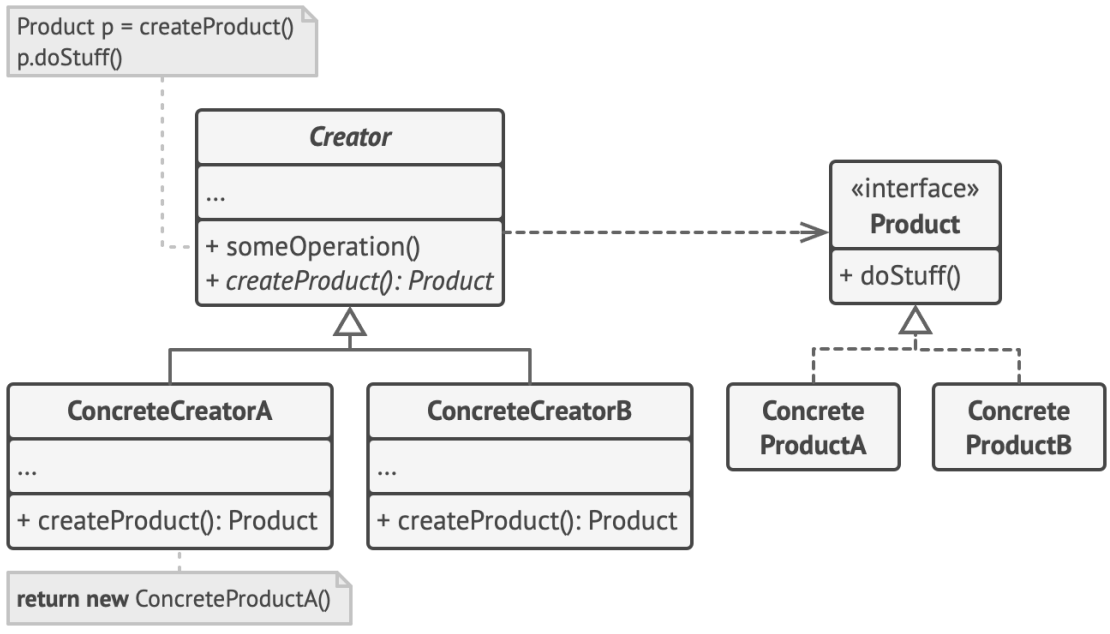
Pizza p = new Pizza.Builder().tamanio("grande")
        .ingrediente("muzzarella").ingrediente("jamón").bordeRelleno(true).build();
```

Roles en el ejemplo: Pizza es el Product; Pizza.Builder es el ConcreteBuilder; el código que encadena las llamadas hace de Director.

- **Cuándo usarlo:** cuando el objeto tiene muchos parámetros, varios opcionales, o se arma por etapas. Evita los constructores con diez argumentos.
- **Contexto:** consultas SQL, documentos HTML o PDF, objetos de configuración. StringBuilder y @Builder de Lombok son ejemplos conocidos.

Factory Method (Método de fabricación)

Define una interfaz para crear un objeto, pero deja que las subclases decidan qué clase instanciar.



fuente: Refactoring Guru

Ejemplo en Java: logística con distintos medios de transporte.

```
interface Transporte { void entregar(String carga); }
class Camion implements Transporte {
    public void entregar(String c) { System.out.println("Entrega por ruta: " + c); }
}
class Barco implements Transporte {
    public void entregar(String c) { System.out.println("Entrega por mar: " + c); }
}

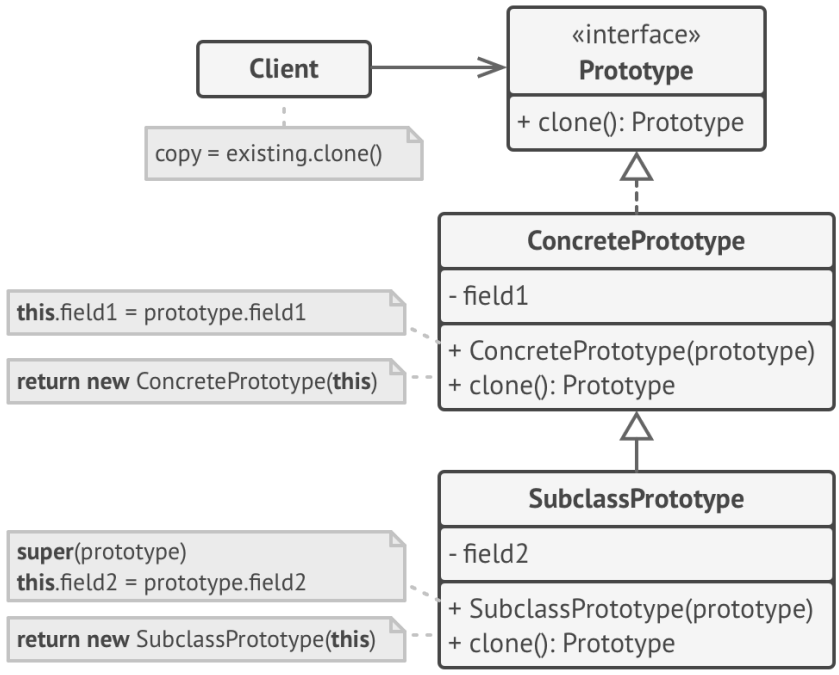
abstract class Logistica {
    abstract Transporte crearTransporte();        // factory method
    void planificarEntrega(String carga) {
        crearTransporte().entregar(carga);        // usa el producto sin conocer su
clase
    }
}
class LogisticaTerrestre extends Logistica {
    Transporte crearTransporte() { return new Camion(); }
}
class LogisticaMaritima extends Logistica {
    Transporte crearTransporte() { return new Barco(); }
}
```

Roles en el ejemplo: Logistica es el Creator; LogisticaTerrestre y LogisticaMaritima son los ConcreteCreator; Transporte es el Product; Camion y Barco son los ConcreteProduct.

- **Cuándo usarlo:** cuando una clase no puede anticipar qué objetos debe crear, o querés que sean las subclases quienes lo decidan.
- **Contexto:** frameworks, envío de notificaciones (email, SMS, push), parsers para distintos formatos.

Prototype (Prototipo)

Crea nuevos objetos copiando una instancia existente, sin depender de su clase concreta.



fuente: Refactoring Guru

Ejemplo en Java: enemigos de un videojuego que se duplican a partir de uno ya configurado.

```
class Enemigo implements Cloneable {
    String tipo;
    int vida;
    List<String> armas = new ArrayList<>();

    @Override
    public Enemigo clone() {
        try {
            Enemigo copia = (Enemigo) super.clone();
            copia.armas = new ArrayList<>(armas);   // copia profunda de la lista
            return copia;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }
}

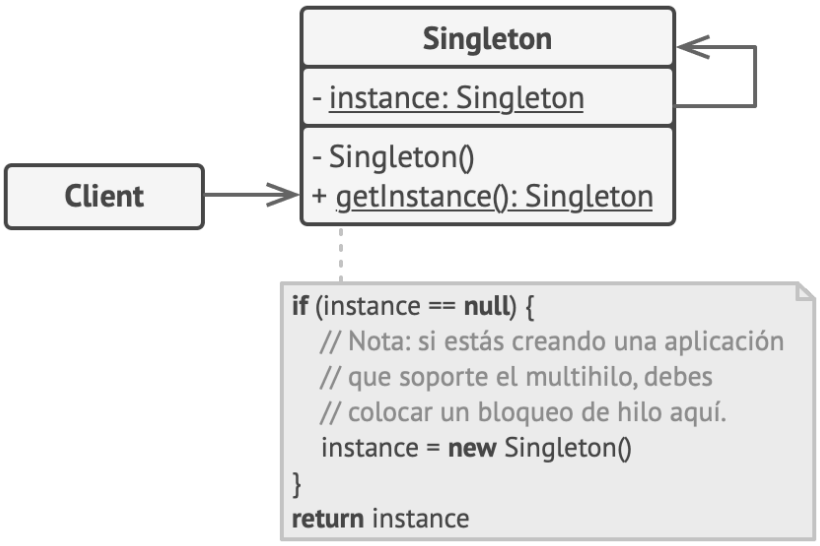
Enemigo orco = new Enemigo();   // se configura una sola vez
Enemigo otroOrco = orco.clone();
```

Roles en el ejemplo: Cloneable de Java cumple el rol de interfaz Prototype; Enemigo es el ConcretePrototype; el código que llama a clone() es el Client.

- **Cuándo usarlo:** cuando crear un objeto desde cero es costoso, o necesitás copias de un objeto ya configurado.
- **Contexto:** plantillas de documentos, personajes o enemigos en un videojuego.

Singleton (Instancia única)

Garantiza que una clase tenga una única instancia y ofrece un punto de acceso global a ella.



fuente: Refactoring Guru

Ejemplo en Java: un logger compartido por toda la aplicación.

```
public class Logger {
    private static final Logger INSTANCIA = new Logger();   // se crea al cargar la
clase

    private Logger() { }                                     // nadie más puede
instanciarla

    public static Logger getInstance() { return INSTANCIA; }

    public void log(String mensaje) { System.out.println("[LOG] " + mensaje); }
}

Logger.getInstance().log("Aplicación iniciada");
```

Roles en el ejemplo: Logger es el Singleton; getInstance() es el método estático de acceso.

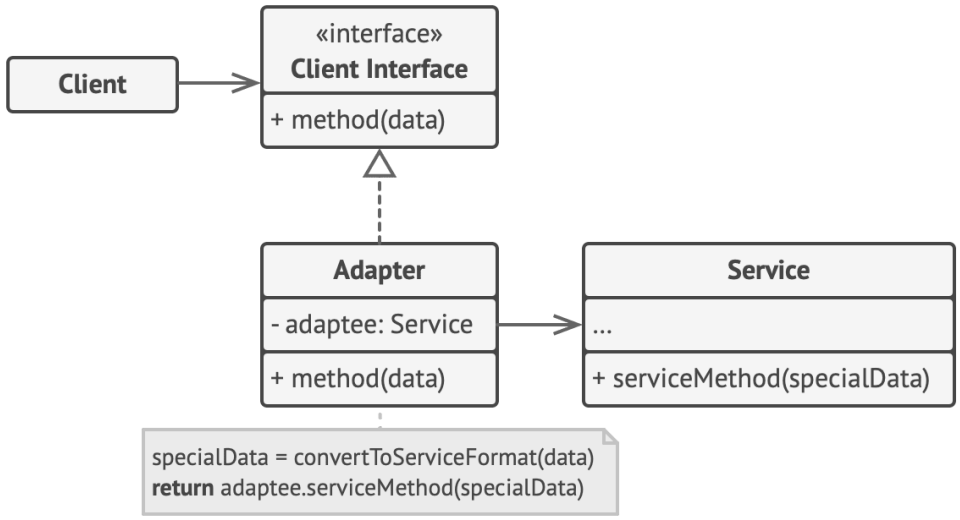
- **Cuándo usarlo:** cuando debe existir exactamente una instancia accesible por todos.
- **Contexto:** configuración global, logger, caché, pool de conexiones. En Spring, los beans son singleton por defecto.

Patrones estructurales

Resuelven el problema de combinar clases y objetos en estructuras más grandes, de forma flexible.

Adapter (Adaptador)

Convierte la interfaz de una clase en otra que el cliente espera, para que clases incompatibles puedan trabajar juntas.



fuente: Refactoring Guru

Ejemplo en Java: una pasarela de pagos de terceros con interfaz distinta a la de la aplicación.

```
interface ProcesadorPago { boolean pagar(double monto); }

// Clase de terceros: no se puede modificar y trabaja en centavos
class PasarelaExterna {
    boolean realizarCobro(int montoEnCentavos) { /* ... */ return true; }
}

class PasarelaAdapter implements ProcesadorPago {
    private final PasarelaExterna pasarela = new PasarelaExterna();

    public boolean pagar(double monto) {
        return pasarela.realizarCobro((int) (monto * 100));   // traduce la llamada
    }
}

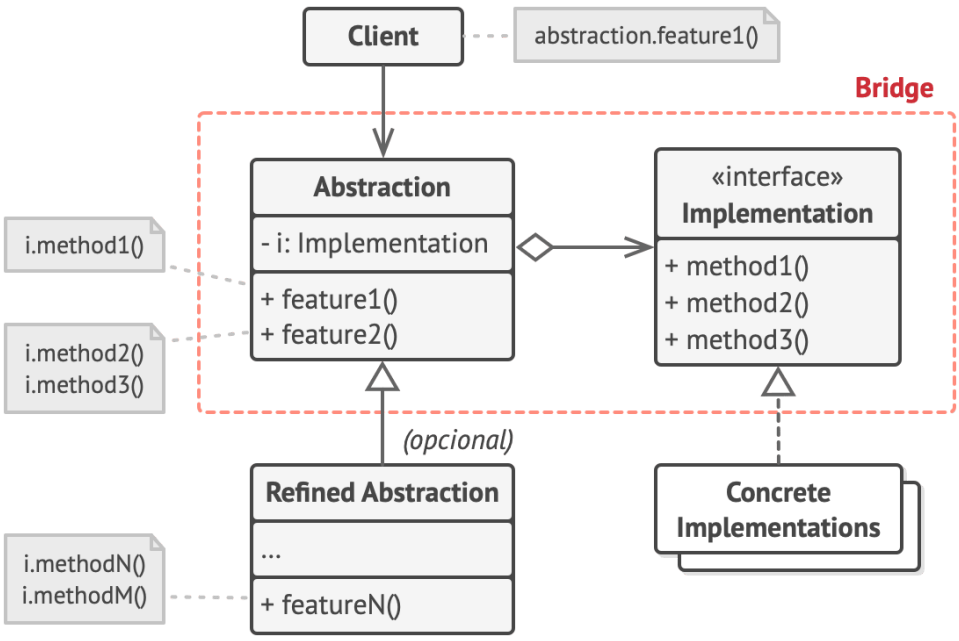
ProcesadorPago pago = new PasarelaAdapter();
pago.pagar(1500.50);
```

Roles en el ejemplo: ProcesadorPago es el Target; PasarelaExterna es el Adaptee; PasarelaAdapter es el Adapter; el código que paga es el Client.

- **Cuándo usarlo:** cuando querés reutilizar una clase existente cuya interfaz no coincide con la que espera tu código.
- **Contexto:** integrar una API de terceros o código legado que no podés modificar.

Bridge (Puente)

Desacopla una abstracción de su implementación para que las dos puedan variar de forma independiente.



fuente: Refactoring Guru

Ejemplo en Java: mensajes (normal o urgente) que se envían por distintos canales.

```
interface Canal { void enviar(String texto); }
class CanalEmail implements Canal {
    public void enviar(String t) { System.out.println("Email: " + t); }
}
class CanalSMS implements Canal {
    public void enviar(String t) { System.out.println("SMS: " + t); }
}

abstract class Mensaje {                          // abstracción
    protected final Canal canal;
    Mensaje(Canal canal) { this.canal = canal; }
    abstract void enviar(String texto);
}
class MensajeNormal extends Mensaje {
    MensajeNormal(Canal c) { super(c); }
    void enviar(String t) { canal.enviar(t); }
}
class MensajeUrgente extends Mensaje {            // abstracción refinada
    MensajeUrgente(Canal c) { super(c); }
    void enviar(String t) { canal.enviar("[URGENTE] " + t); }
}
```

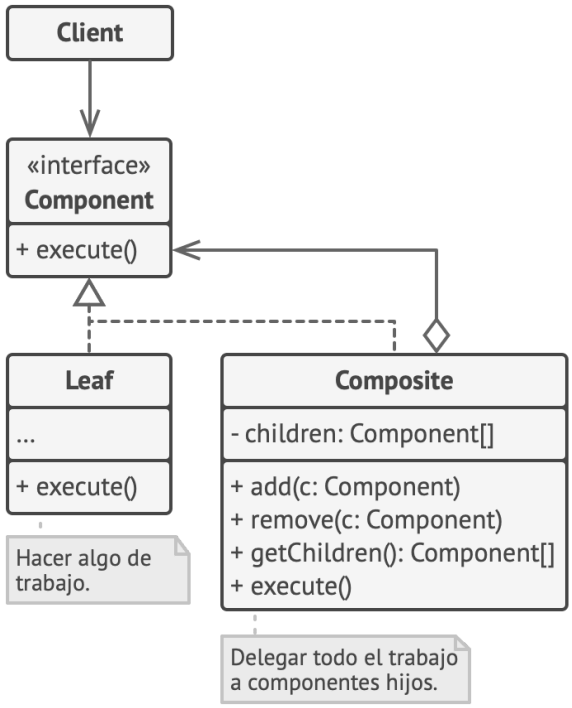
```
new MensajeUrgente(new CanalSMS()).enviar("Servidor caído");
```

Roles en el ejemplo: Mensaje es la Abstraction; MensajeNormal y MensajeUrgente son las RefinedAbstraction; Canal es la Implementation; CanalEmail y CanalSMS son las ConcreteImplementation.

- **Cuándo usarlo:** cuando una clase varía en dos dimensiones a la vez y querés evitar la explosión de subclases (N tipos de mensaje por M canales).
- **Contexto:** controles remotos y dispositivos, formas geométricas y motores de render, JDBC.

Composite (Objeto compuesto)

Compone objetos en estructuras de árbol y permite tratar de la misma forma a un objeto individual y a un grupo.



fuente: Refactoring Guru

Ejemplo en Java: un sistema de archivos donde una carpeta puede contener archivos y otras carpetas.

```
interface Nodo { long tamanio(); }

class Archivo implements Nodo {
    private final long bytes;
    Archivo(long bytes) { this.bytes = bytes; }
```

```
    public long tamanio() { return bytes; }
}

class Carpeta implements Nodo {
    private final List<Nodo> hijos = new ArrayList<>();
    void agregar(Nodo n) { hijos.add(n); }
    public long tamanio() {
        return hijos.stream().mapToLong(Nodo::tamanio).sum();
    }
}

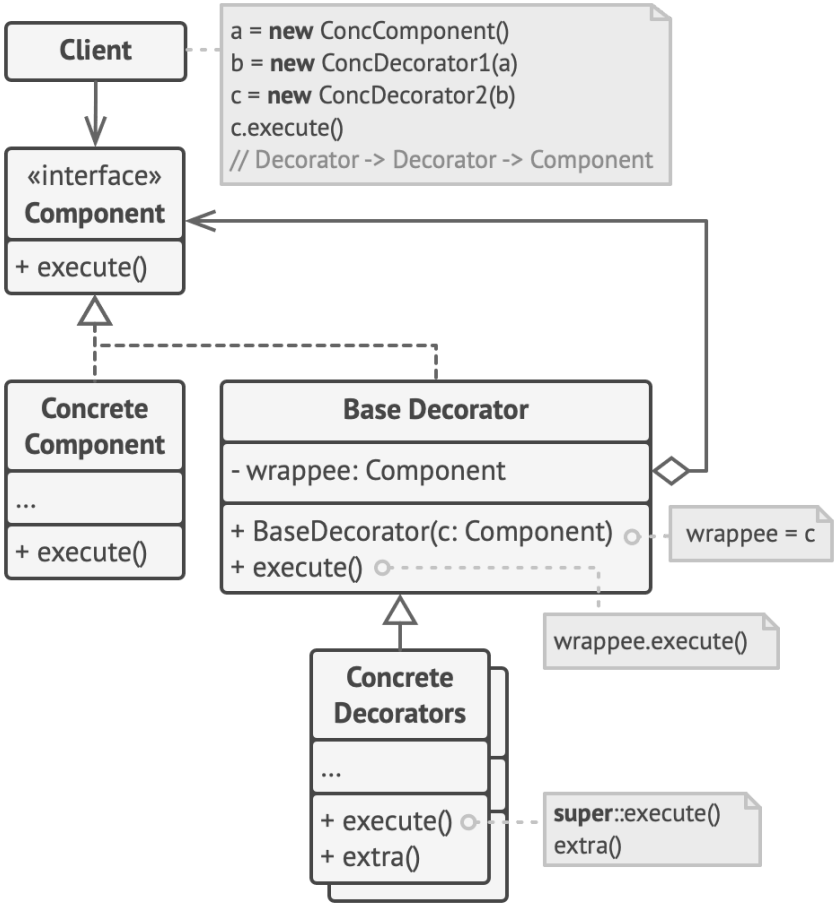
Carpeta raiz = new Carpeta();
raiz.agregar(new Archivo(2_000));
Carpeta fotos = new Carpeta();
fotos.agregar(new Archivo(5_000));
raiz.agregar(fotos);           // una carpeta dentro de otra
raiz.tamanio();                // 7000
```

Roles en el ejemplo: Nodo es el Component; Archivo es la Leaf; Carpeta es el Composite.

- **Cuándo usarlo:** cuando modelás jerarquías parte-todo y querés que el cliente trate igual a los elementos simples y a los compuestos.
- **Contexto:** sistemas de archivos (archivo y carpeta), menús con submenús, componentes de interfaz gráfica.

Decorator (Decorador)

Agrega responsabilidades a un objeto de forma dinámica, como alternativa flexible a la herencia.



fuente: Refactoring Guru

Ejemplo en Java: transformaciones de texto que se pueden combinar.

```
interface Texto { String contenido(); }

class TextoSimple implements Texto {
    private final String texto;
    TextoSimple(String t) { texto = t; }
    public String contenido() { return texto; }
}

abstract class TextoDecorador implements Texto {
    protected final Texto envuelto;
    TextoDecorador(Texto e) { envuelto = e; }
}
class Mayusculas extends TextoDecorador {
    Mayusculas(Texto e) { super(e); }
    public String contenido() { return envuelto.contenido().toUpperCase(); }
}
class ConExclamacion extends TextoDecorador {
    ConExclamacion(Texto e) { super(e); }
    public String contenido() { return envuelto.contenido() + "!"; }
```

```
}

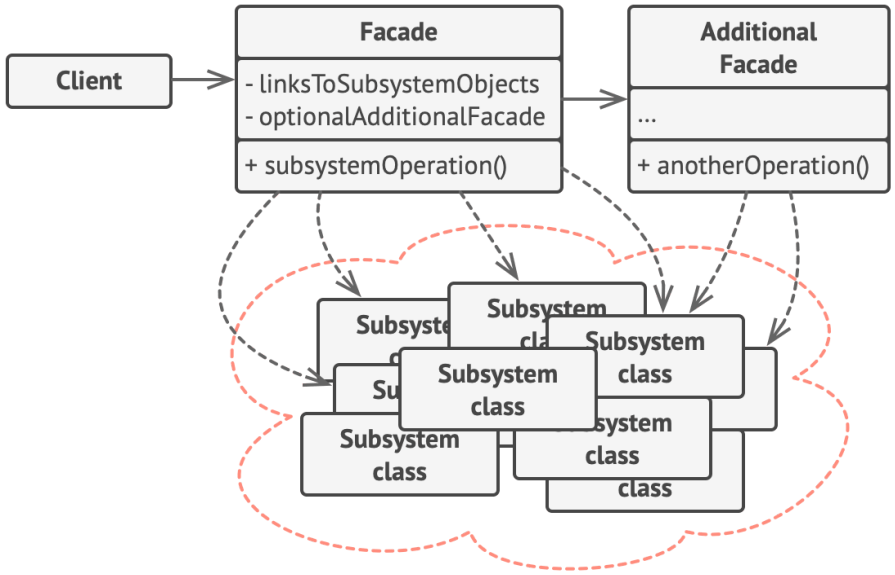
Texto t = new ConExclamacion(new Mayusculas(new TextoSimple("hola")));
t.contenido();   // "HOLA!"
```

Roles en el ejemplo: Texto es el Component; TextoSimple es el ConcreteComponent; TextoDecorador es el Decorator base; Mayusculas y ConExclamacion son los ConcreteDecorator.

- **Cuándo usarlo:** cuando querés sumar comportamiento a objetos individuales, de forma combinable y en tiempo de ejecución, sin una subclase por cada combinación.
- **Contexto:** los streams de Java (new BufferedReader(new FileReader(...))), extras de un pedido, filtros.

Facade (Fachada)

Provee una interfaz unificada y simple para un conjunto de interfaces de un subsistema complejo.



fuente: Refactoring Guru

Ejemplo en Java: una compra online que involucra inventario, pagos y envíos.

```
class CompraFacade {
    private final Inventario inventario = new Inventario();
    private final Pagos pagos = new Pagos();
    private final Envios envios = new Envios();

    boolean comprar(String producto, double monto, String direccion) {
        if (!inventario.hayStock(producto)) return false;
        if (!pagos.cobrar(monto)) return false;
```

```
        inventario.descontar(producto);
        envios.despachar(producto, direccion);
        return true;
    }
}

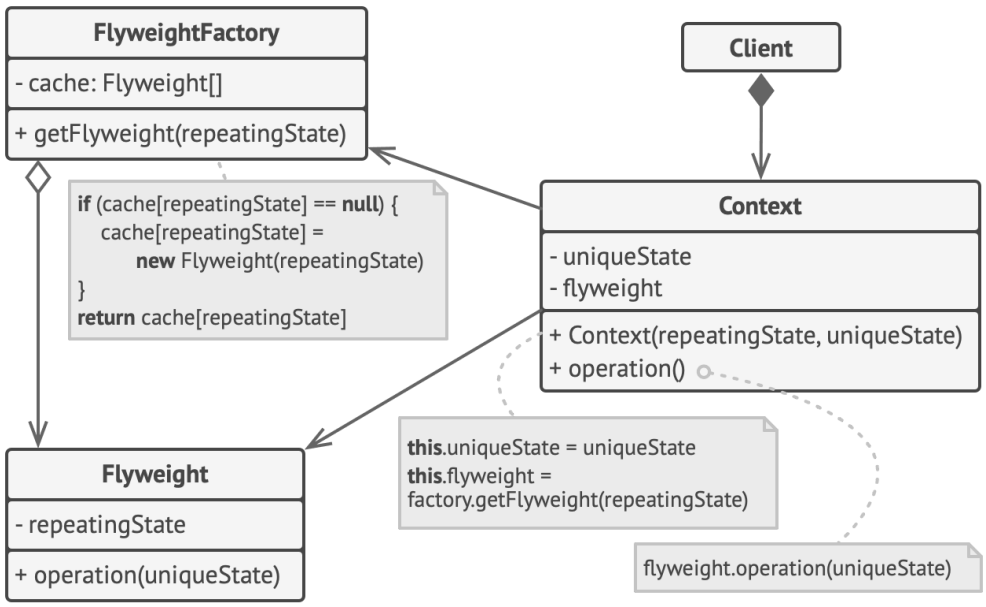
// El cliente llama a un solo método
new CompraFacade().comprar("Notebook", 850000, "Mendoza");
```

Roles en el ejemplo: CompraFacade es la Facade; Inventario, Pagos y Envios son las clases del subsistema.

- **Cuándo usarlo:** cuando querés un punto de entrada simple a un subsistema complejo, o reducir el acoplamiento entre capas.
- **Contexto:** una capa de servicio que orquesta varios repositorios, el SDK de una librería compleja.

Flyweight (Peso ligero)

Comparte eficientemente una gran cantidad de objetos pequeños para reducir el consumo de memoria.



fuente: Refactoring Guru

Ejemplo en Java: los caracteres de un editor de texto comparten sus estilos.

```
class Estilo {                                    // estado intrínseco: compartido e
inmutable
    final String fuente;
    final int tamanio;
    Estilo(String fuente, int tamanio) { this.fuente = fuente; this.tamanio =
```

```
tamanio; }
}

class FabricaEstilos {
    private static final Map<String, Estilo> cache = new HashMap<>();
    static Estilo get(String fuente, int tamanio) {
        return cache.computeIfAbsent(fuente + tamanio, k -> new Estilo(fuente,
tamanio));
    }
}

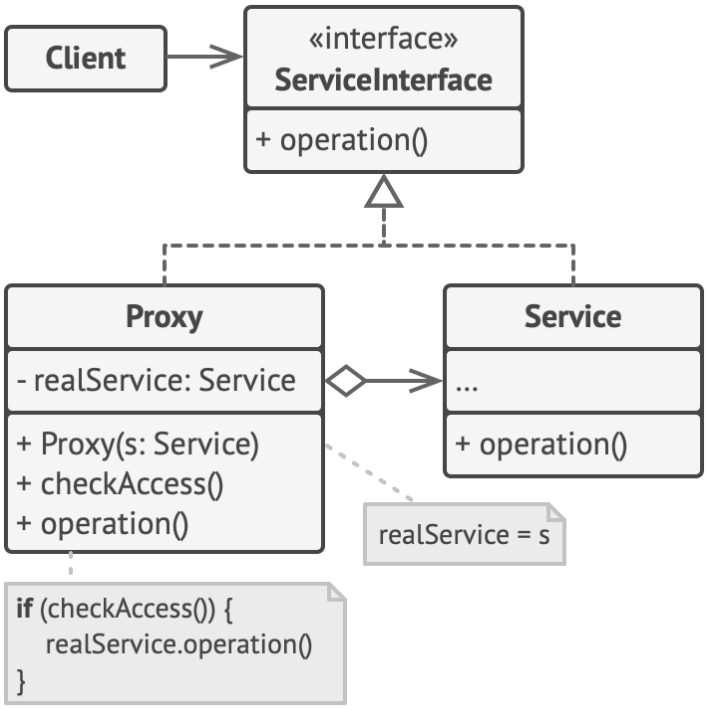
class Caracter {                                  // estado extrínseco: símbolo y
posición
    char simbolo;
    int posicion;
    Estilo estilo;
}
// Un documento de 100.000 caracteres usa solo unos pocos objetos Estilo
```

Roles en el ejemplo: Estilo es el Flyweight; FabricaEstilos es la FlyweightFactory; Caracter es el Context que guarda el estado extrínseco.

- **Cuándo usarlo:** cuando hace falta una enorme cantidad de objetos parecidos y la memoria es un problema.
- **Contexto:** caracteres de un editor de texto, árboles o partículas en un videojuego, el String pool de Java.

Proxy (Apoderado)

Provee un sustituto de otro objeto para controlar el acceso a él.



fuente: Refactoring Guru

Ejemplo en Java: un proxy de protección que verifica permisos antes de leer un documento.

```
interface Documento { String leer(); }

class DocumentoReal implements Documento {
    public String leer() { return "Contenido confidencial"; }
}

class DocumentoProxy implements Documento {      // proxy de protección
    private final DocumentoReal real = new DocumentoReal();
    private final String rol;
    DocumentoProxy(String rol) { this.rol = rol; }

    public String leer() {
        if (!rol.equals("ADMIN")) throw new SecurityException("Sin permiso");
        return real.leer();
    }
}

new DocumentoProxy("ADMIN").leer();    // permitido
new DocumentoProxy("INVITADO").leer(); // lanza SecurityException
```

Roles en el ejemplo: Documento es el Subject (la interfaz de servicio); DocumentoReal es el RealSubject; DocumentoProxy es el Proxy.

- **Cuándo usarlo:** cuando necesitás controlar el acceso a un objeto: carga diferida, permisos, caché o acceso remoto.

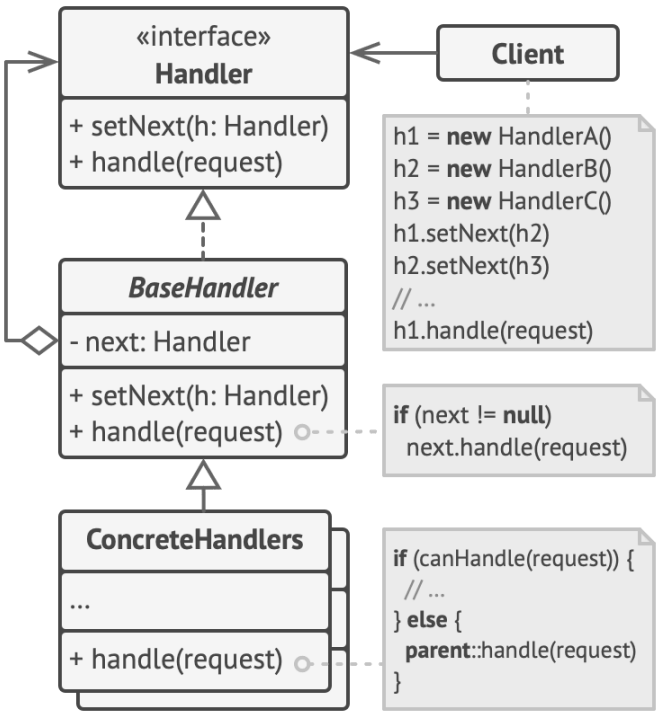
- **Contexto:** carga lazy de entidades en Hibernate, Spring AOP (@Transactional usa proxies).

Patrones de comportamiento

Resuelven el problema de repartir responsabilidades y coordinar la comunicación entre objetos.

Chain of Responsibility (Cadena de responsabilidad)

Pasa una solicitud por una cadena de manejadores hasta que alguno la atiende, desacoplando al emisor del receptor.



fuente: Refactoring Guru

Ejemplo en Java: soporte técnico por niveles según la gravedad del ticket.

```
abstract class Soporte {
    protected Soporte siguiente;
    Soporte setSiguiente(Soporte s) { siguiente = s; return s; }
    abstract void atender(Ticket t);
}
class Nivel1 extends Soporte {
    void atender(Ticket t) {
        if (t.gravedad() <= 1) System.out.println("Resuelve el nivel 1");
        else if (siguiente != null) siguiente.atender(t);
    }
```

```
}
// Nivel2 (gravedad <= 2) y Nivel3 se escriben igual

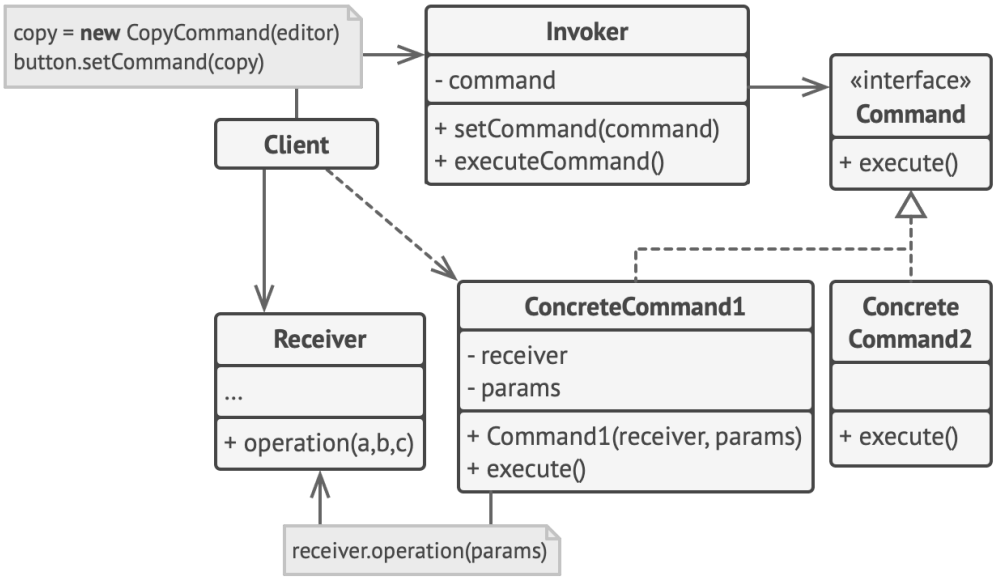
Soporte cadena = new Nivel1();
cadena.setSiguiente(new Nivel2()).setSiguiente(new Nivel3());
cadena.atender(new Ticket(3));   // pasa por los niveles hasta llegar al 3
```

Roles en el ejemplo: Soporte es el Handler; Nivel1, Nivel2 y Nivel3 son los ConcreteHandler; el código que arma la cadena y envía el ticket es el Client.

- **Cuándo usarlo:** cuando más de un objeto puede atender una solicitud y no se sabe de antemano cuál.
- **Contexto:** filtros de servlets y de Spring Security, aprobaciones según el monto, validaciones encadenadas.

Command (Comando)

Encapsula una solicitud como un objeto, para poder parametrizarla, encolarla, registrarla y deshacerla.



fuente: Refactoring Guru

Ejemplo en Java: un editor de texto con función deshacer.

```
interface Comando { void ejecutar(); void deshacer(); }

class Editor { StringBuilder texto = new StringBuilder(); }   // receptor

class Escribir implements Comando {
    private final Editor editor;
```

```
    private final String frase;
    Escribir(Editor e, String f) { editor = e; frase = f; }
    public void ejecutar() { editor.texto.append(frase); }
    public void deshacer() {
        editor.texto.setLength(editor.texto.length() - frase.length());
    }
}

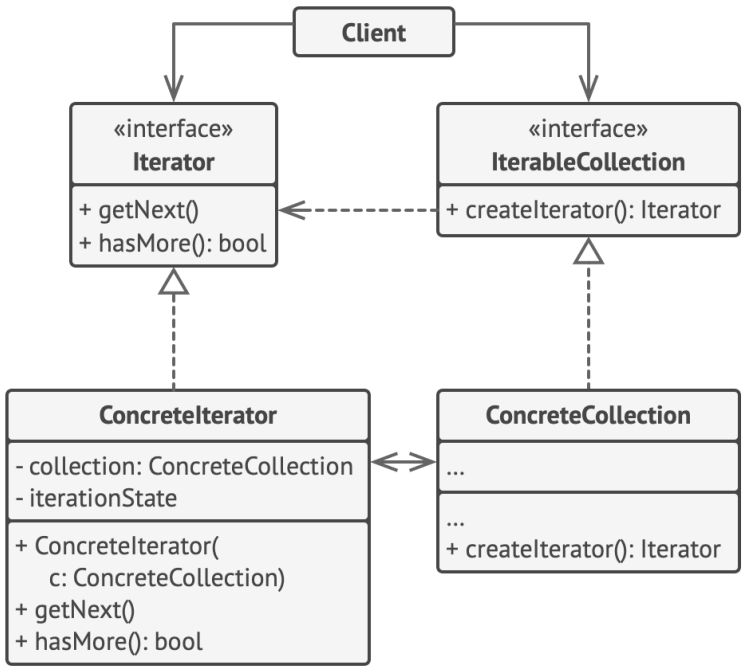
class Historial {                                              // invocador
    private final Deque<Comando> pila = new ArrayDeque<>();
    void correr(Comando c) { c.ejecutar(); pila.push(c); }
    void deshacer()        { if (!pila.isEmpty()) pila.pop().deshacer(); }
}
```

Roles en el ejemplo: Comando es la interfaz Command; Escribir es el ConcreteCommand; Editor es el Receiver; Historial es el Invoker.

- **Cuándo usarlo:** cuando querés tratar una acción como un objeto: ponerla en una cola, registrarla, o permitir deshacer y rehacer.
- **Contexto:** editores con Ctrl+Z, colas de tareas, macros.

Iterator (Iterador)

Da acceso secuencial a los elementos de una colección sin exponer su representación interna.



fuente: Refactoring Guru

Ejemplo en Java: una playlist que se recorre de la última canción a la primera.

```
class Playlist implements Iterable<String> {          // colección
    private final List<String> canciones = new ArrayList<>();
    void agregar(String c) { canciones.add(c); }

    public Iterator<String> iterator() {              // iterador en orden inverso
        return new Iterator<>() {
            private int pos = canciones.size() - 1;
            public boolean hasNext() { return pos >= 0; }
            public String next()     { return canciones.get(pos--); }
        };
    }
}

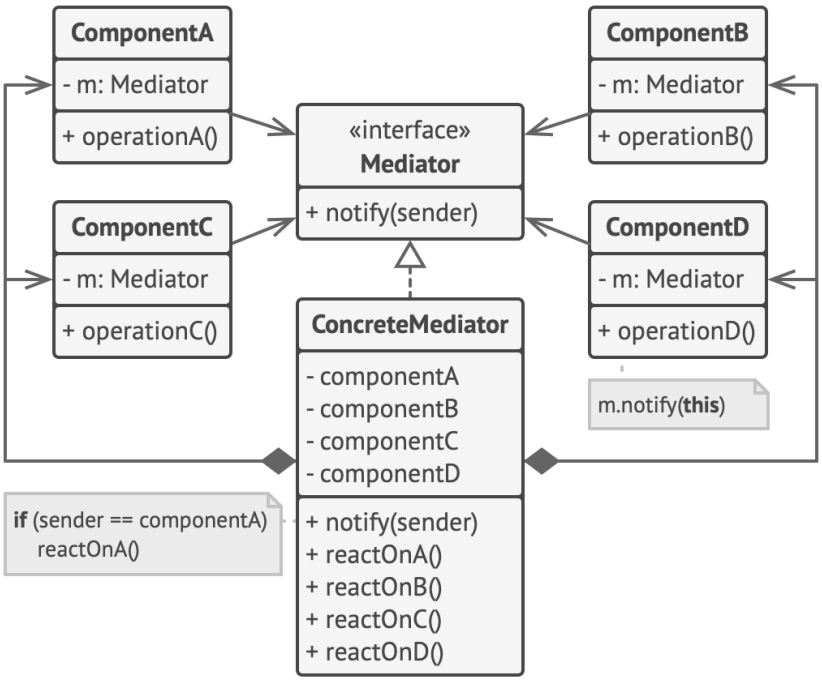
for (String c : playlist) System.out.println(c);      // el cliente no ve la lista
interna
```

Roles en el ejemplo: Iterable es la interfaz Aggregate; Playlist es el ConcreteAggregate; Iterator es la interfaz Iterator; la clase anónima que devuelve iterator() es el ConcreteIterator.

- **Cuándo usarlo:** cuando querés recorrer una colección sin exponer cómo está armada, o necesitás varias formas de recorrerla.
- **Contexto:** las colecciones de Java, recorrido de árboles, paginación de resultados.

Mediator (Mediador)

Define un objeto que encapsula cómo interactúan otros objetos, evitando que se refieran entre sí directamente.



fuente: Refactoring Guru

Ejemplo en Java: una sala de chat que reparte los mensajes entre usuarios.

```
interface Mediador { void enviar(String mensaje, Usuario origen); }

class SalaChat implements Mediador {
    private final List<Usuario> usuarios = new ArrayList<>();
    void registrar(Usuario u) { usuarios.add(u); }

    public void enviar(String mensaje, Usuario origen) {
        for (Usuario u : usuarios)
            if (u != origen) u.recibir(origen.nombre + ": " + mensaje);
    }
}

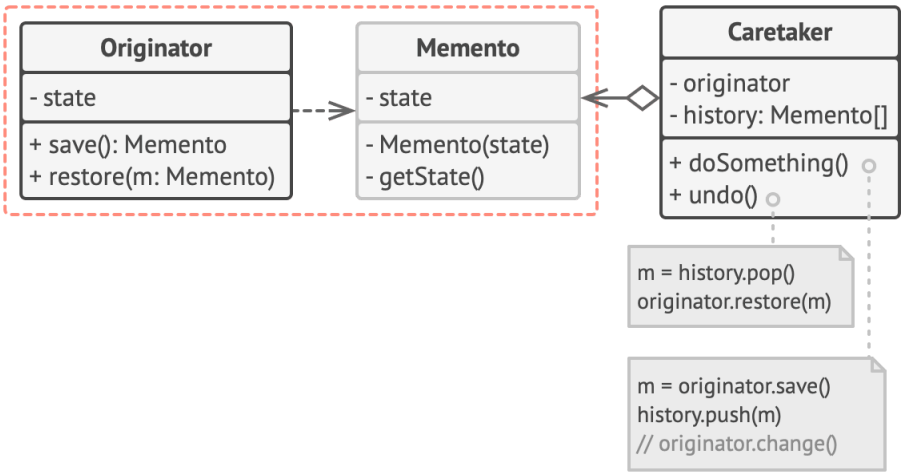
class Usuario {
    final String nombre;
    private final Mediador sala;
    Usuario(String nombre, Mediador sala) { this.nombre = nombre; this.sala = sala; }
    void decir(String m)   { sala.enviar(m, this); }   // no conoce a los demás
usuarios
    void recibir(String m) { System.out.println(nombre + " recibe -> " + m); }
}
```

Roles en el ejemplo: Mediador es la interfaz Mediator; SalaChat es el ConcreteMediator; Usuario es el Colleague.

- **Cuándo usarlo:** cuando muchos objetos se comunican de forma enredada y querés centralizar esa coordinación.
- **Contexto:** formularios donde habilitar un botón depende de varios campos, torre de control aéreo, sala de chat.

Memento (Recuerdo)

Captura y guarda el estado interno de un objeto, sin violar su encapsulamiento, para poder restaurarlo después.



fuente: Refactoring Guru

Ejemplo en Java: guardar y cargar una partida de un videojuego.

```
class Juego {                                        // originador
    private int nivel = 1, vida = 100;
    void avanzar() { nivel++; vida -= 20; }

    Partida guardar()      { return new Partida(nivel, vida); }
    void cargar(Partida p) { nivel = p.nivel; vida = p.vida; }

    static class Partida {                           // memento: opaco para los demás
        private final int nivel, vida;
        private Partida(int n, int v) { nivel = n; vida = v; }
    }
}

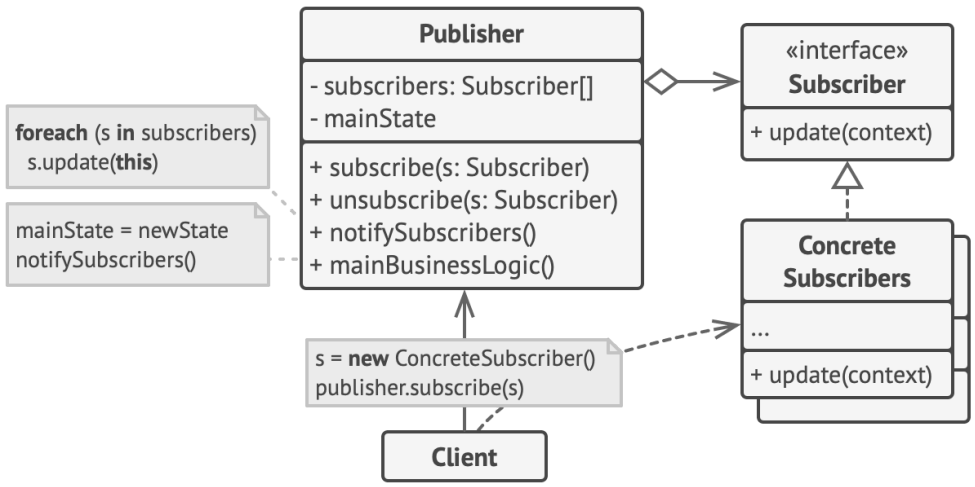
Juego juego = new Juego();
Juego.Partida punto = juego.guardar();               // el cuidador la conserva
juego.avanzar();
juego.cargar(punto);                                 // vuelve al estado guardado
```

Roles en el ejemplo: Juego es el Originator; Juego.Partida es el Memento; el código cliente que conserva el objeto Partida cumple el rol de Caretaker.

- **Cuándo usarlo:** cuando necesitás guardar el estado de un objeto para volver atrás, sin exponer sus atributos internos.
- **Contexto:** deshacer en editores, guardado de partidas, rollback de operaciones.

Observer (Observador)

Define una dependencia uno-a-muchos: cuando un objeto cambia de estado, se notifica y actualiza automáticamente a sus dependientes.



fuente: Refactoring Guru

Ejemplo en Java: un canal de videos que avisa a sus suscriptores.

```
interface Suscriptor { void notificar(String video); }

class Canal {                                          // sujeto
    private final List<Suscriptor> suscriptores = new ArrayList<>();
    void suscribir(Suscriptor s)   { suscriptores.add(s); }
    void desuscribir(Suscriptor s) { suscriptores.remove(s); }
    void publicar(String video) {
        suscriptores.forEach(s -> s.notificar(video));   // avisa a todos
    }
}

class Usuario implements Suscriptor {
    private final String nombre;
    Usuario(String n) { nombre = n; }
    public void notificar(String video) { System.out.println(nombre + " ve: " +
video); }
}

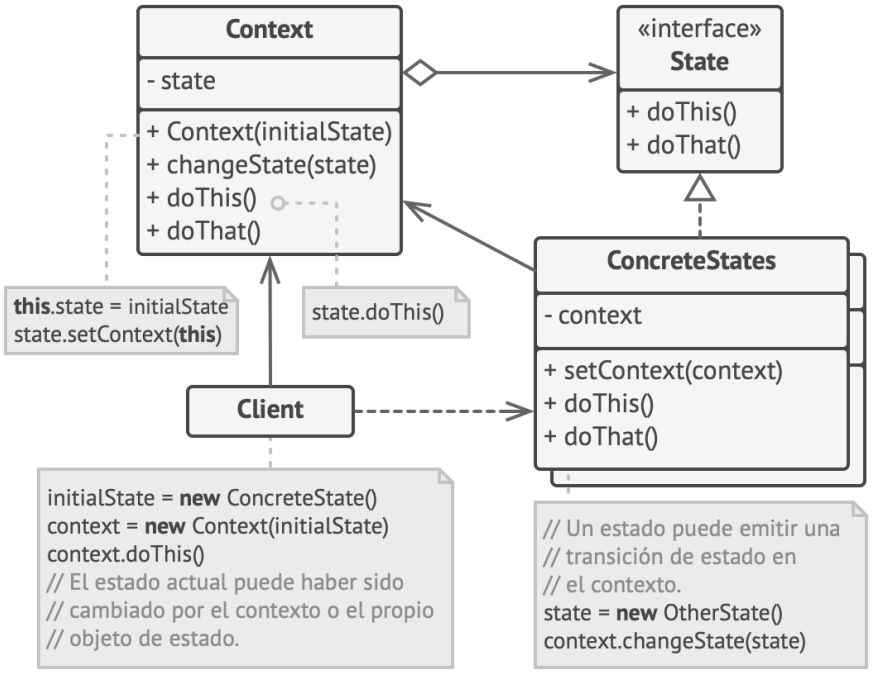
Canal canal = new Canal();
canal.suscribir(new Usuario("Ana"));
canal.suscribir(new Usuario("Luis"));
canal.publicar("Patrones GoF");
```

Roles en el ejemplo: Canal es el Subject (Publisher); Suscriptor es la interfaz Observer; Usuario es el ConcreteObserver.

- **Cuándo usarlo:** cuando el cambio en un objeto debe reflejarse en otros que no se conocen de antemano.
- **Contexto:** eventos de interfaz gráfica, MVC (el modelo notifica a las vistas), eventos de Spring (ApplicationEvent).

State (Estado)

Permite que un objeto cambie su comportamiento cuando cambia su estado interno; parece que cambia de clase.



Ejemplo en Java: un reproductor de música con el botón de play.

```
interface Estado { void presionarPlay(Reproductor r); }

class Detenido implements Estado {
    public void presionarPlay(Reproductor r) {
        System.out.println("Reproduciendo");
        r.setEstado(new Reproduciendo());
    }
}
class Reproduciendo implements Estado {
    public void presionarPlay(Reproductor r) {
        System.out.println("En pausa");
        r.setEstado(new Pausado());
    }
}
class Pausado implements Estado {
    public void presionarPlay(Reproductor r) {
        System.out.println("Reproduciendo");
        r.setEstado(new Reproduciendo());
    }
}

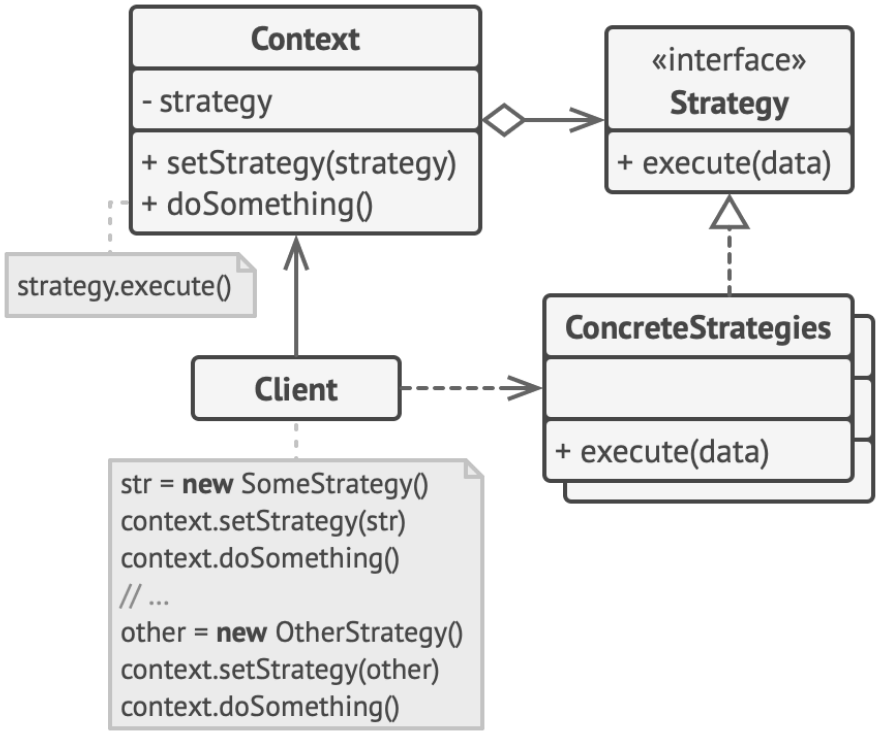
class Reproductor {                                    // contexto
    private Estado estado = new Detenido();
    void setEstado(Estado e) { estado = e; }
    void presionarPlay()     { estado.presionarPlay(this); }
}
```

Roles en el ejemplo: Reproductor es el Context; Estado es la interfaz State; Detenido, Reproduciendo y Pausado son los ConcreteState.

- **Cuándo usarlo:** cuando el comportamiento de un objeto depende de su estado y el código está lleno de if o switch que lo consultan.
- **Contexto:** ciclo de vida de un pedido (pendiente, enviado, entregado), reproductor de música, conexión de red.

Strategy (Estrategia)

Define una familia de algoritmos, encapsula cada uno y los hace intercambiables en tiempo de ejecución.



fuente: Refactoring Guru

Ejemplo en Java: distintas formas de calcular el costo de envío de un pedido.

```
interface CostoEnvio { double calcular(double kilos); }

class EnvioEstandar implements CostoEnvio {
    public double calcular(double kilos) { return 500 + kilos * 100; }
}
class EnvioExpress implements CostoEnvio {
    public double calcular(double kilos) { return 1200 + kilos * 250; }
}

class Pedido {                                          // contexto
    private CostoEnvio estrategia;
    Pedido(CostoEnvio e) { estrategia = e; }
    void setEstrategia(CostoEnvio e) { estrategia = e; }  // se puede cambiar en
```

```
ejecución
    double costoEnvio(double kilos)  { return estrategia.calcular(kilos); }
}

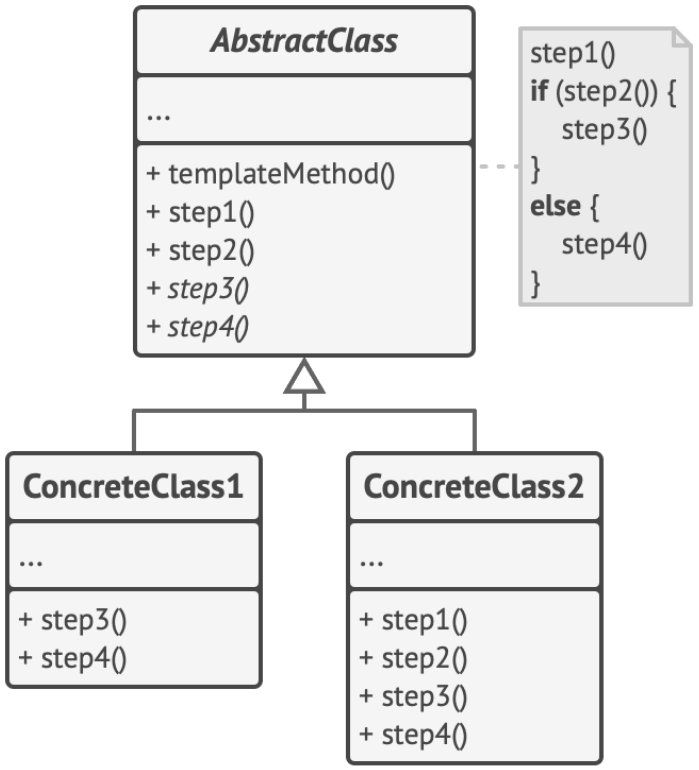
new Pedido(new EnvioExpress()).costoEnvio(2);   // 1700
```

Roles en el ejemplo: Pedido es el Context; CostoEnvio es la interfaz Strategy; EnvioEstandar y EnvioExpress son las ConcreteStrategy.

- **Cuándo usarlo:** cuando hay varias formas de hacer lo mismo y querés elegirlas en tiempo de ejecución, evitando condicionales largos.
- **Contexto:** Comparator para ordenar, cálculo de descuentos o impuestos, medios de pago.

Template Method (Método plantilla)

Define el esqueleto de un algoritmo en una clase base y deja que las subclases redefinan algunos pasos sin cambiar su estructura.



fuente: Refactoring Guru

Ejemplo en Java: preparación de bebidas calientes.

```
abstract class Bebida {
    public final void preparar() {            // método plantilla: el orden no cambia
```

```
        hervirAgua();
        infusionar();
        servir();
        if (quiereAzucar()) agregarAzucar();  // gancho opcional
    }
    void hervirAgua()  { System.out.println("Hirviendo agua"); }
    void servir()      { System.out.println("Sirviendo en la taza"); }
    void agregarAzucar() { System.out.println("Agregando azúcar"); }
    abstract void infusionar();               // paso que cambia en cada subclase
    boolean quiereAzucar() { return false; }
}

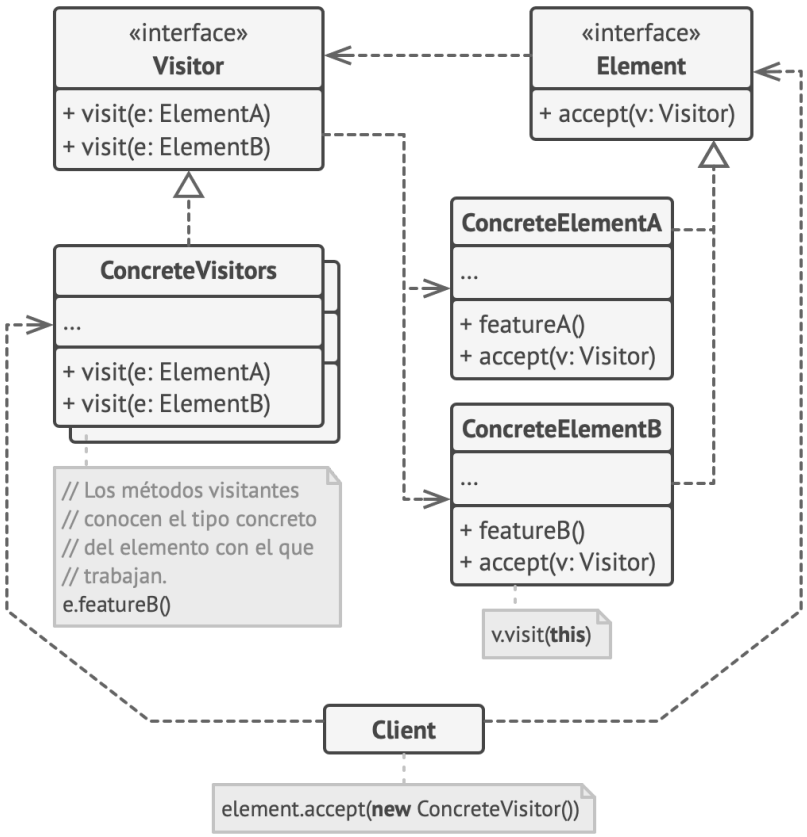
class Te extends Bebida {
    void infusionar() { System.out.println("Infusionando el té"); }
}
class Cafe extends Bebida {
    void infusionar()      { System.out.println("Filtrando el café"); }
    boolean quiereAzucar() { return true; }
}
```

Roles en el ejemplo: Bebida es la AbstractClass; preparar() es el templateMethod; infusionar() es un paso abstracto; quiereAzucar() es un gancho (hook); Te y Cafe son las ConcreteClass.

- **Cuándo usarlo:** cuando varias clases comparten la estructura de un algoritmo y solo difieren en algunos pasos.
- **Contexto:** frameworks con ganchos de ciclo de vida, procesos ETL, generación de reportes, JdbcTemplate de Spring.

Visitor (Visitante)

Representa una operación sobre los elementos de una estructura de objetos, permitiendo agregar operaciones nuevas sin modificar las clases de esos elementos.



Ejemplo en Java: cálculo de impuestos según el tipo de producto.

```
interface Visitante {
    double visitar(Libro l);
    double visitar(Electronico e);
}
interface Producto { double aceptar(Visitante v); }

class Libro implements Producto {
    double precio;
    public double aceptar(Visitante v) { return v.visitar(this); }
}
class Electronico implements Producto {
    double precio;
    public double aceptar(Visitante v) { return v.visitar(this); }
}

class CalculadorImpuestos implements Visitante {
    public double visitar(Libro l)       { return l.precio * 0.05; }
    public double visitar(Electronico e) { return e.precio * 0.21; }
}

double impuesto = producto.aceptar(new CalculadorImpuestos());
```

Roles en el ejemplo: Visitante es la interfaz Visitor; CalculadorImpuestos es el ConcreteVisitor; Producto es la interfaz Element; Libro y Electronico son los ConcreteElement.

- **Cuándo usarlo:** cuando la estructura de clases es estable pero necesitás agregar operaciones nuevas con frecuencia sin tocarlas.
- **Contexto:** compiladores (recorrer un árbol sintáctico), exportar un árbol de documentos a distintos formatos.